Amazon Braket

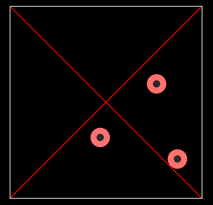
Quantum computing on AWS

Aoyu

Applied Scientist

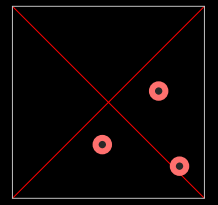
Quantum Technologies at AWS

Amazon Braket



Advanced Solutions Lab

AWS Center for Quantum Computing

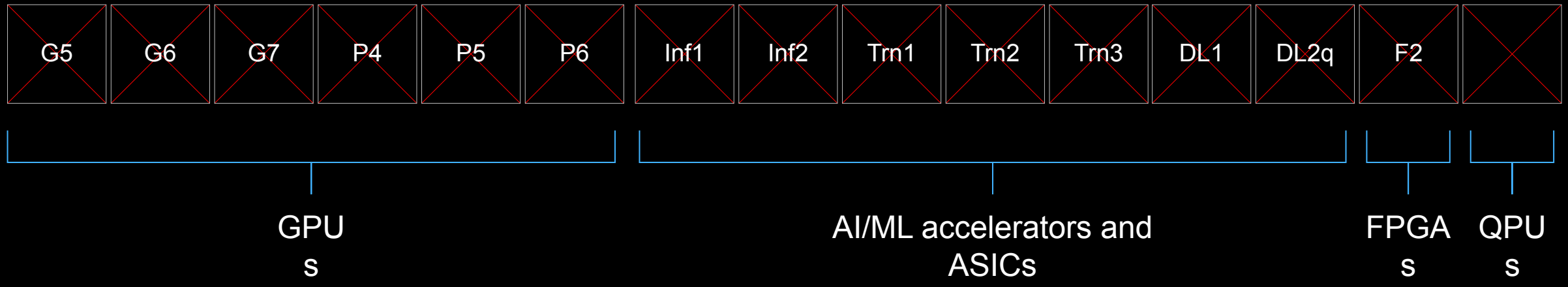


Post-Quantum Cryptography & Quantum Networking

Accelerate quantum computing research

Vision: Quantum as foundational compute technology

Part of our accelerated computing portfolio

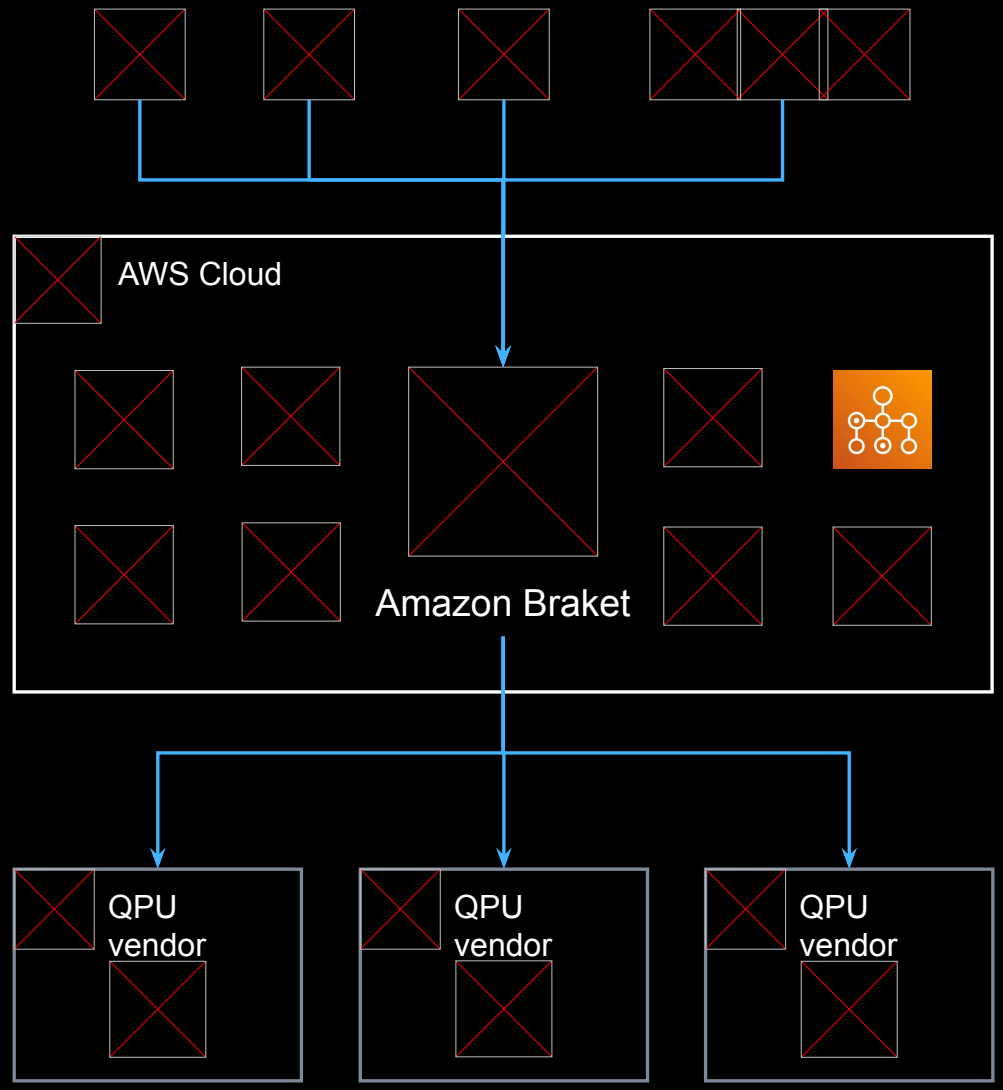


Trainium accelerator Inferentia accelerator	GB200, B200, H200, H100, A100, V100, A10G, T4	Gaudi accelerator	Radeon GPU Xilinx accelerator Xilinx FPGA	Qualcomm Cloud AI 100

Amazon Braket

AWS quantum computing cloud service

- On-demand & dedicated access to quantum hardware from various providers
- Unified user experience across device modalities
- Low-level access and device-native programming
- Fully-managed execution environment for quantum-classical workloads
- AWS cloud integration

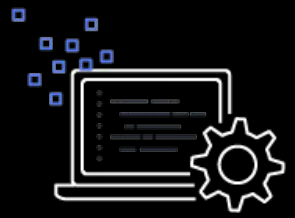


Amazon Braket : Accelerate quantum computing research

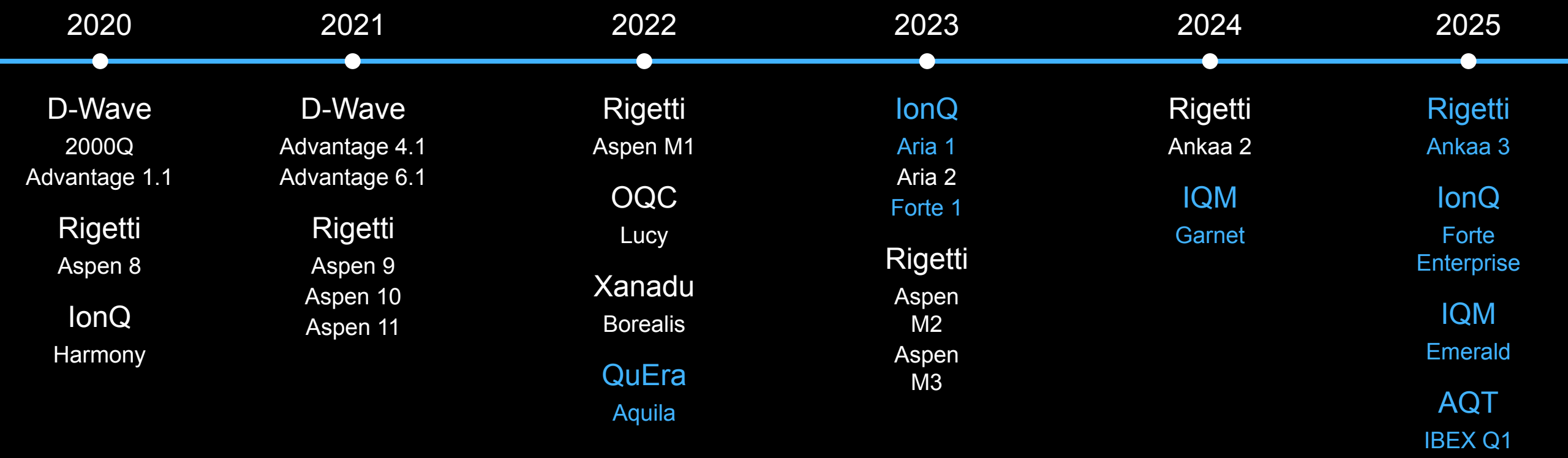
Discovering use cases

Hands-on experimentation

Pushing the boundaries

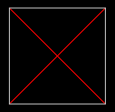


Quantum hardware integrated over time



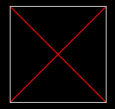
How users interact with Braket

Service APIs



Quantum Tasks

Run quantum programs on on-demand simulators or QPUs

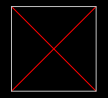


Braket Hybrid Jobs

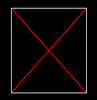
Run hybrid programs combining quantum and classical computing resources

AWS SDKs

Call Amazon Braket from within your applications



Python

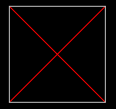


C++

Rust

…and more programming languages

Programming Frameworks



Amazon Braket Python SDK

Pennylane

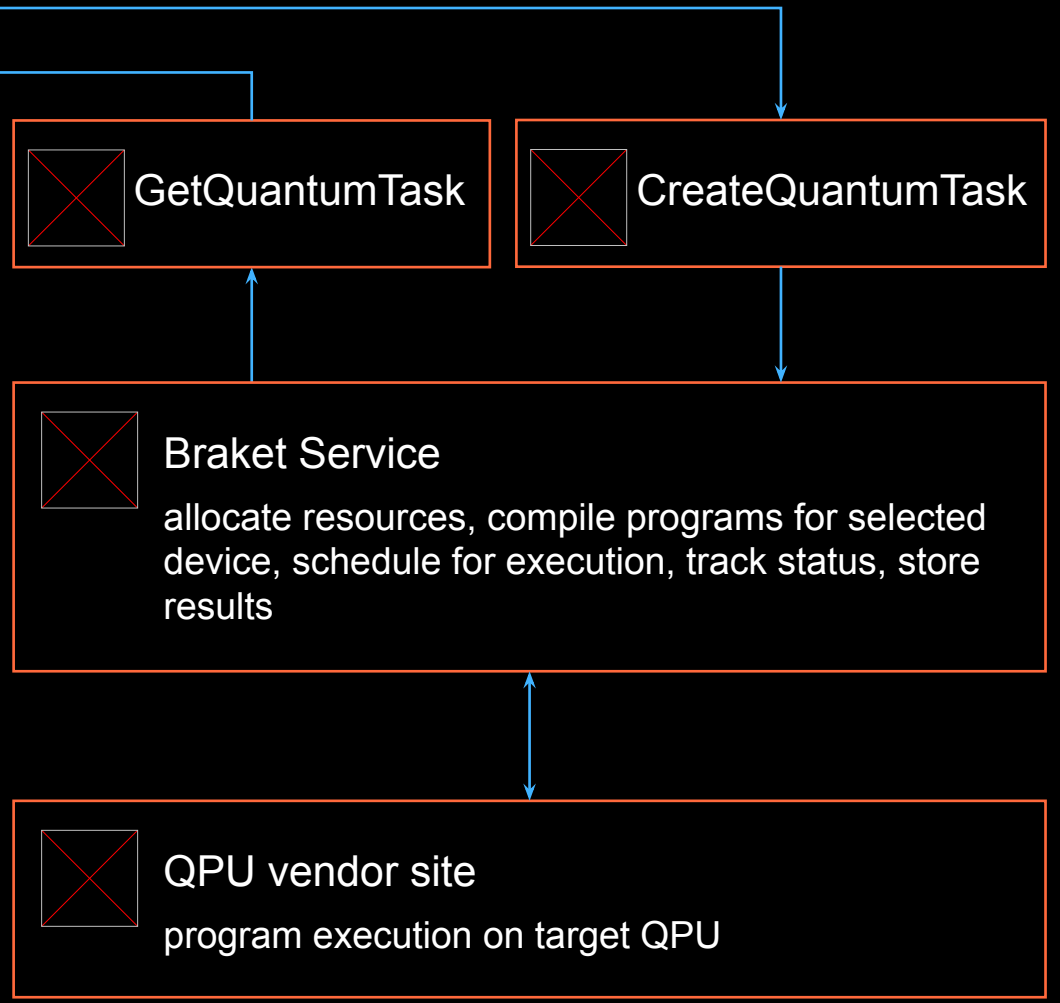
Qiskit

CUDA-Q

…and more 3rd-party tools & programming frameworks supported

Run logical quantum programs with Braket

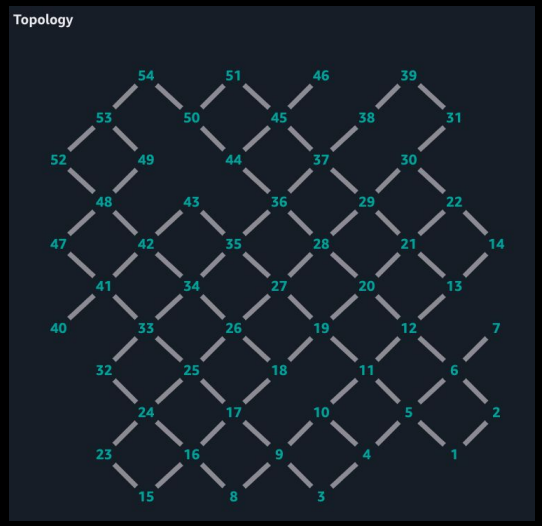
```
~ pip install amazon-braket-sdk
~
~ python
>>> from braket.aws import AwsDevice
>>> from braket.devices import Devices
>>> from braket.circuits import Circuit
>>>
>>> circuit = Circuit().h(0).cnot(0,1)
>>>
>>> qpu = AwsDevice(Devices.IQM.Garnet)
>>>
>>> task = qpu.run(circuit, shots=100)
>>> task.state()
'QUEUED'
>>> task.queue_position()
QuantumTaskQueueInfo(queue_type='Normal', queue_position='1')
>>> task.state()
'COMPLETED'
>>>
>>> result = task.result()
>>> result.additional_metadata.iqmMetadata.compiledProgram
'OPENQASM 3.0;\nbit[2] c;\n#pragma braket verbatim\nbox
{\nprx(0.5*pi,1.5*pi) $49;\nprx(0.5*pi,1.5*pi) $53;\ncz
$49,$53;\nprx(0.5*pi,0.5*pi) $53;\n}\nc[0] = measure
$49;\nc[1] = measure $53;'
```

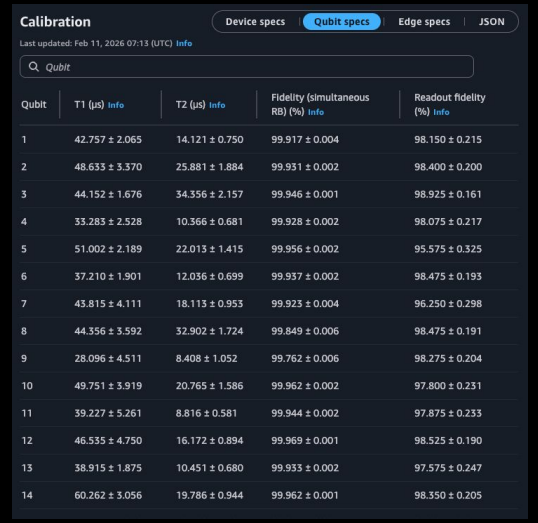


Low-level access and device-native programming

```
>>> qpu = AwsDevice(Devices.IQM.Garnet)
>>>
>>> qpu.properties.paradigm.nativeGateSet
['cz', 'prx', 'cc_prx', 'measure_ff']
>>>
>>> circuit = Circuit().prx(1,0.5*pi,1.5*pi)
>>> circuit.prx(2,0.5*pi,1.5*pi).cz(1,2).prx(2,0.5*pi,0.5*pi)
>>> verbatim_circuit = Circuit().add_verbatim_box(circuit)
>>> task = qpu.run(verbatim_circuit, shots=100,
     disable_qubit_rewiring=True)
```

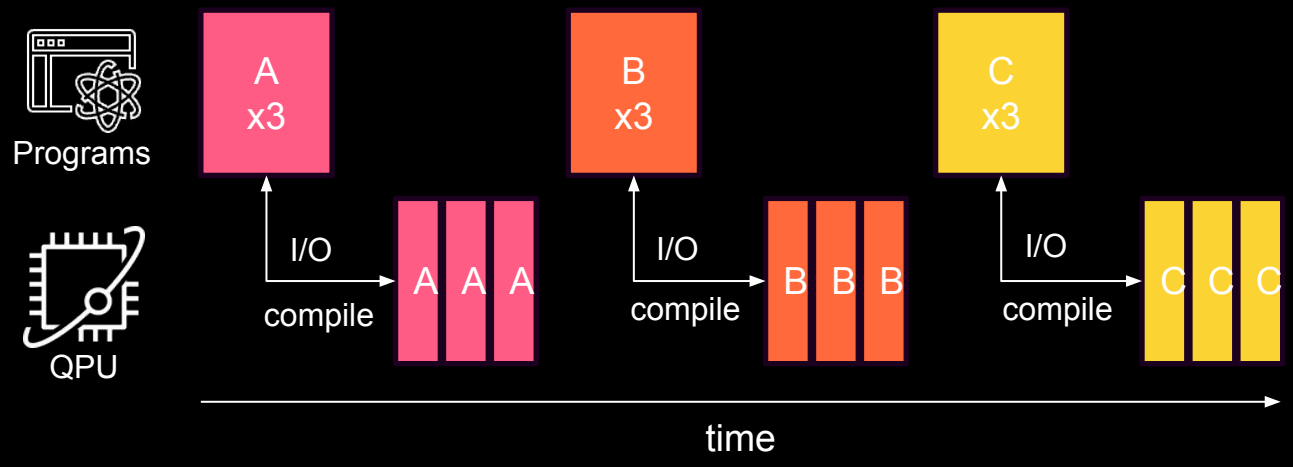
```
>>> from braket.pulse import PulseSequence, GaussianWaveform
>>>
>>> qpu = AwsDevice(Devices.Rigetti.Ankaa3)
>>>
>>> drive_frame = device.frames["Transmon_25_charge_tx"]
>>> readout_frame = device.frames["Transmon_25_readout_rx"]
>>> waveform = GaussianWaveform(FreeParameter("length"),
     FreeParameter("length")*0.25, 0.2, False)
>>> pulse_sequence = PulseSequence().play(drive_frame,
waveform).capture_v0(readout_frame)
>>> task = qpu.run(pulse_sequence(length=12e-9), shots=100)
```





- Access to device properties: qubit count, native gates, connectivity, calibration data…
- Program verbatim circuits executed as defined, without modifications
- Access pulse control on certain devices and define your programs analog pulse sequences

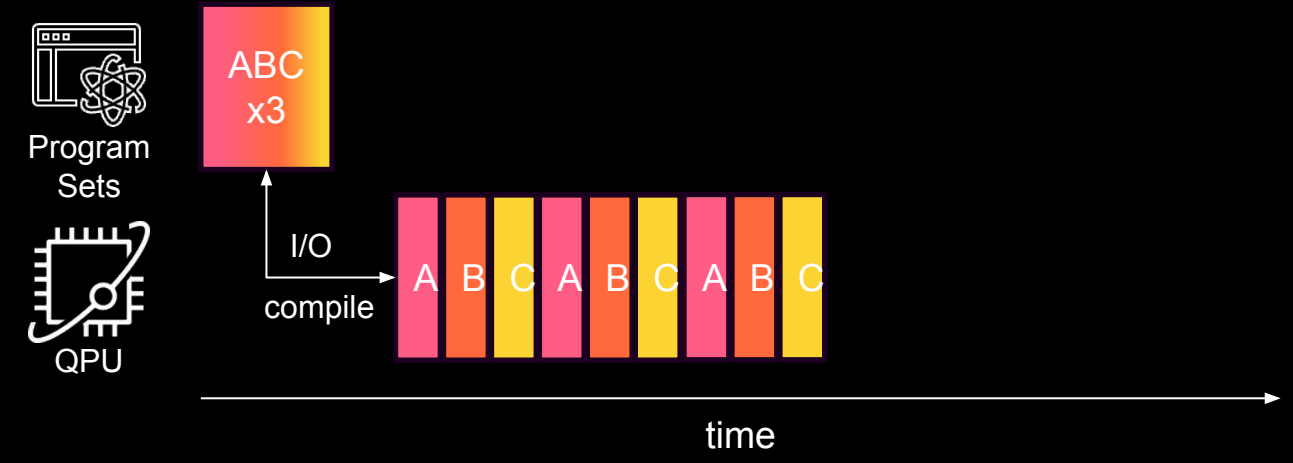
Braket program sets – efficient batch execution



Common workload pattern for

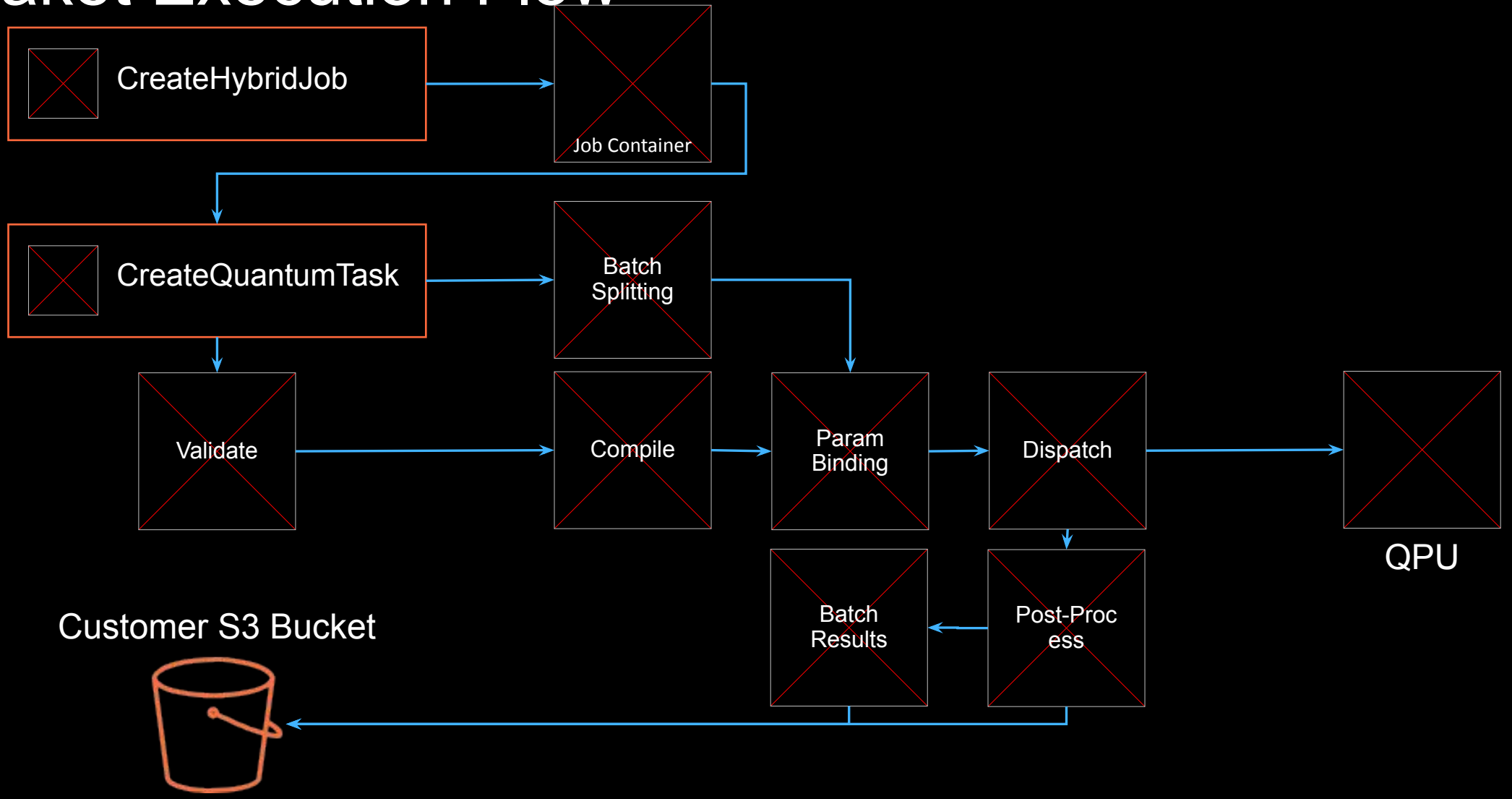
- Hamiltonian estimation
- parameter sweeps
- gradient computations
- …

Program sets

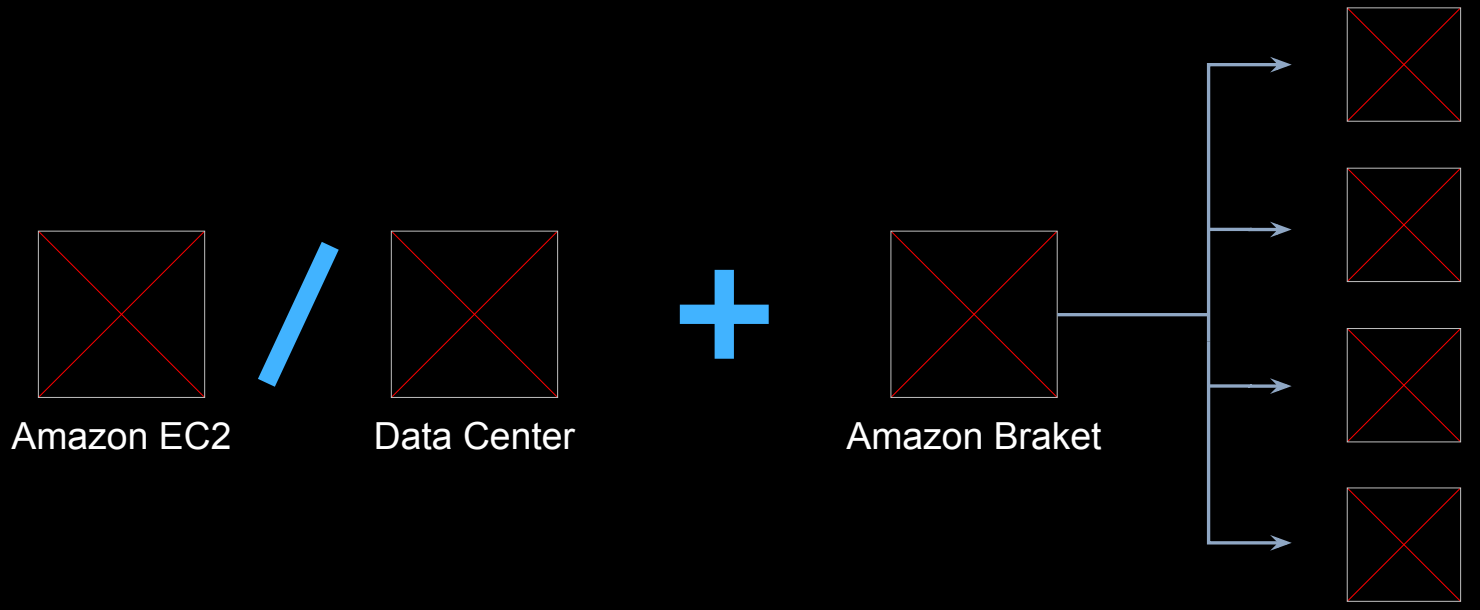


- Submit multiple quantum programs in a single payload
- Compiled to a hardware efficient representation
- End-to-end execution with up to 24x speedup

Braket Execution Flow



Integration of quantum and classical resources



Service-managed execution environment

Cloud-native service integration

Traditional workload management

Amazon Braket Hybrid Jobs

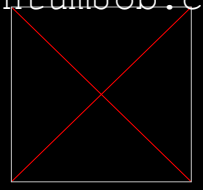
Fully-managed execution environment that makes it easy to run quantum-classical workloads on Braket

```
>>> import pennylane as qml
>>> from braket.jobs import hybrid_job
>>>
>>> @hybrid_job(device=Devices.IonQ.Aria1, local=False)
>>> def qubit_rotation(steps=20, step_size=0.5, shots=1000):
...   device = qml.device("braket.aws.qubit", device_arn=get_job_device_arn(), wires=1, shots=shots)
...
...   @qml.qnode(device)
...   def circuit(params):
...     qml.RX(params[0], wires=0)
...     return qml.expval(qml.PauliZ(0))
...
...   opt = qml.GradientDescentOptimizer(step_size)
...   for i in range(steps):
...      params, expval = opt.step_and_cost(circuit, params)
...
...   return {"theta": params.tolist()[0]}
>>>
>>> job = qubit_rotation()
```

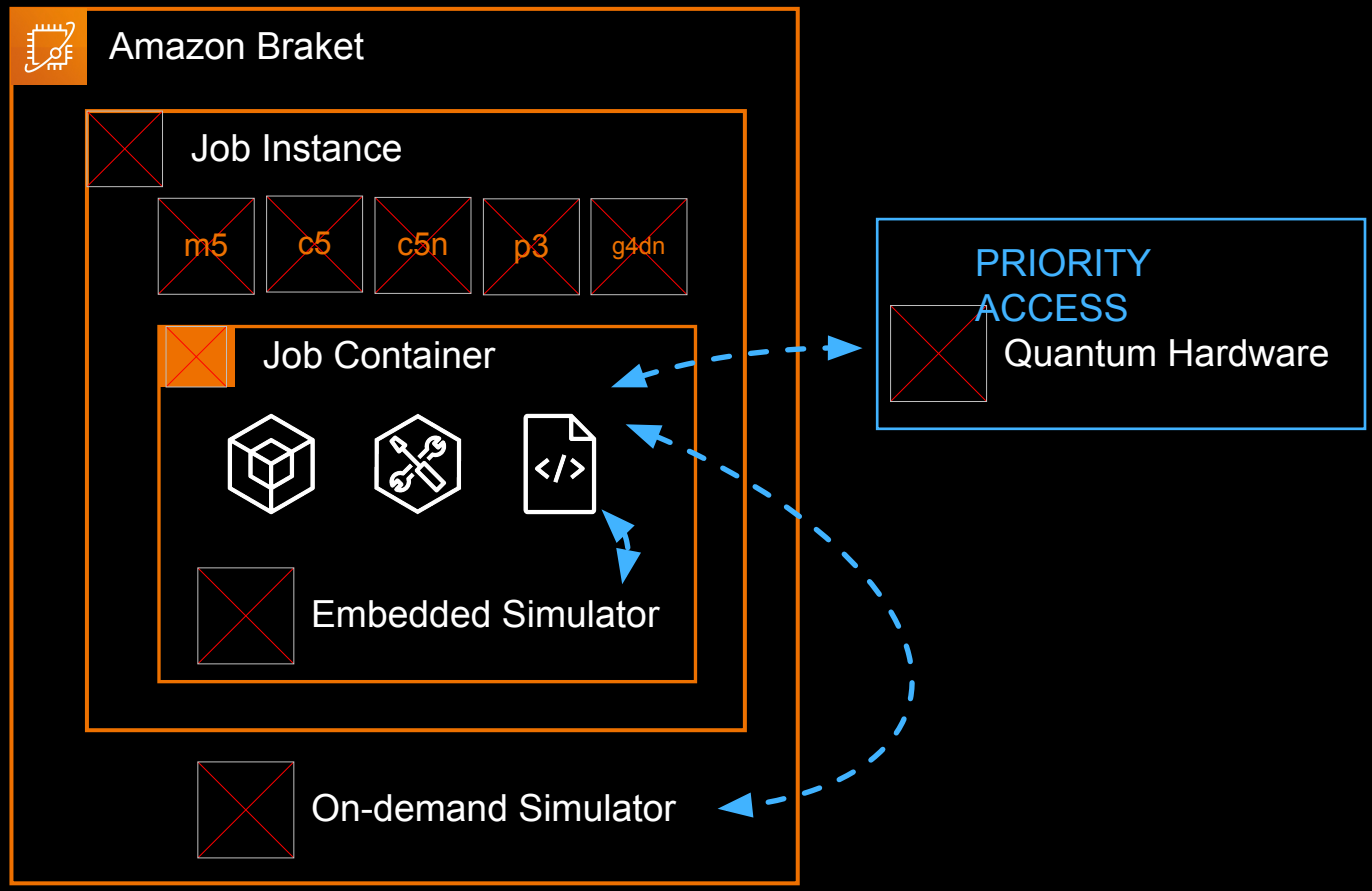
Amazon Braket Hybrid Jobs nuts and bolts

```
from braket.aws import
AwsQuantumJob

job = AwsQuantumJob.create(…)
```



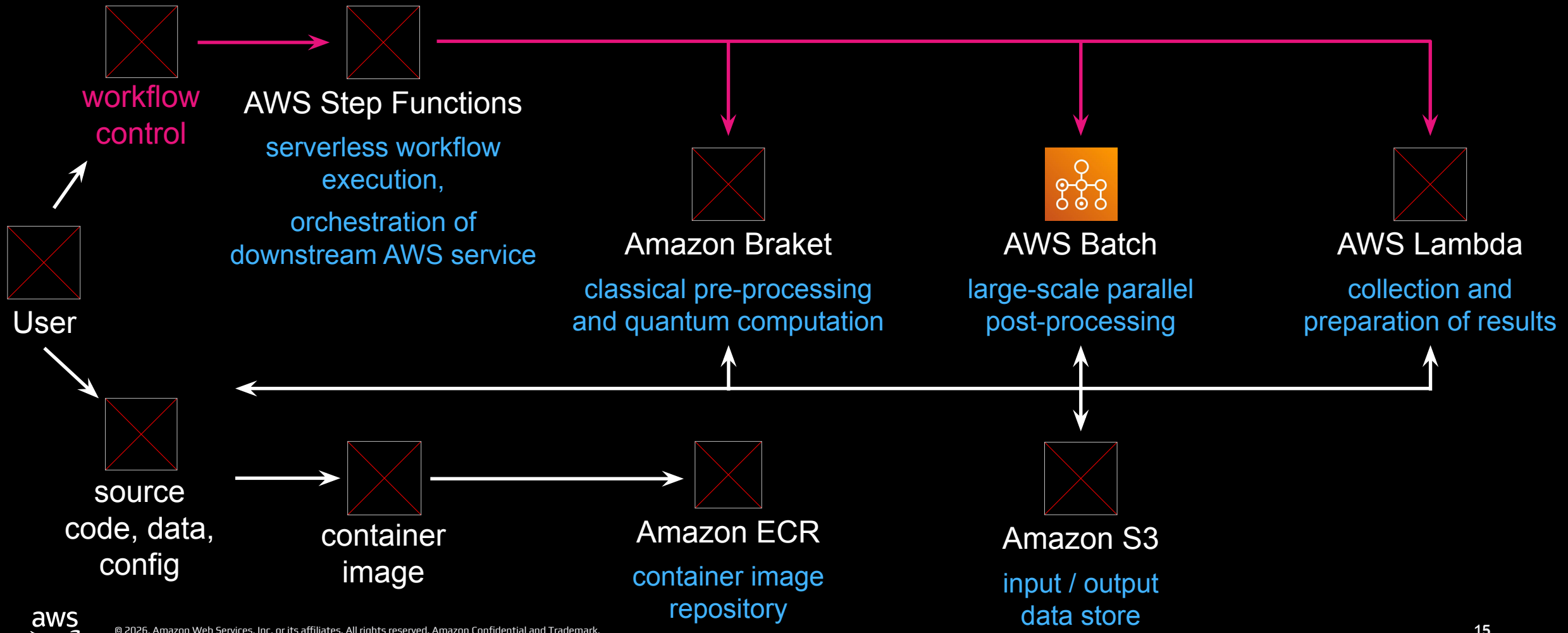
CreateJob API



- Start and provision Job Instance: user-selected classical compute resources
- Run Job Container: based on pre-built or user-provided Docker image
- Execute algorithm code
- Run tasks on QPUs, on-demand simulators, or simulators embedded into job container
- Release compute resources at job completion

Cloud-native compute pipeline

For workflows with classical pre- / post-processing at HPC scale

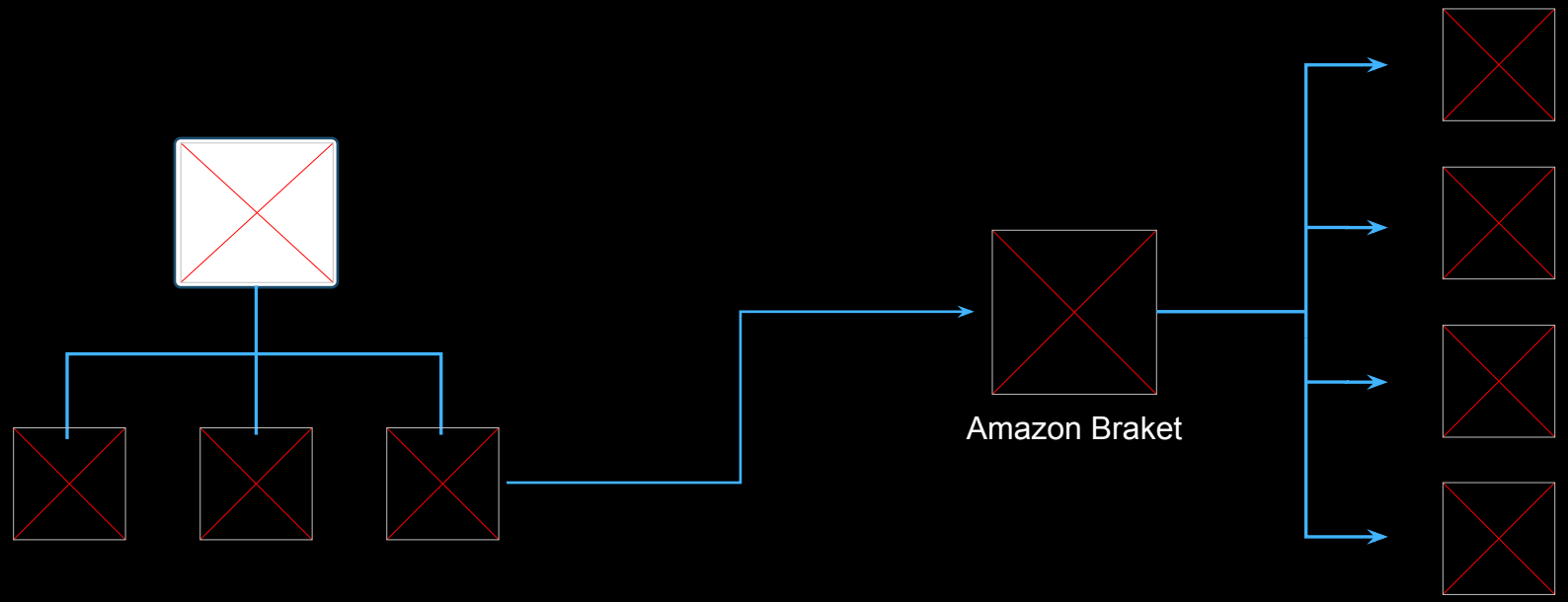


Deploy and run the hybrid workflow

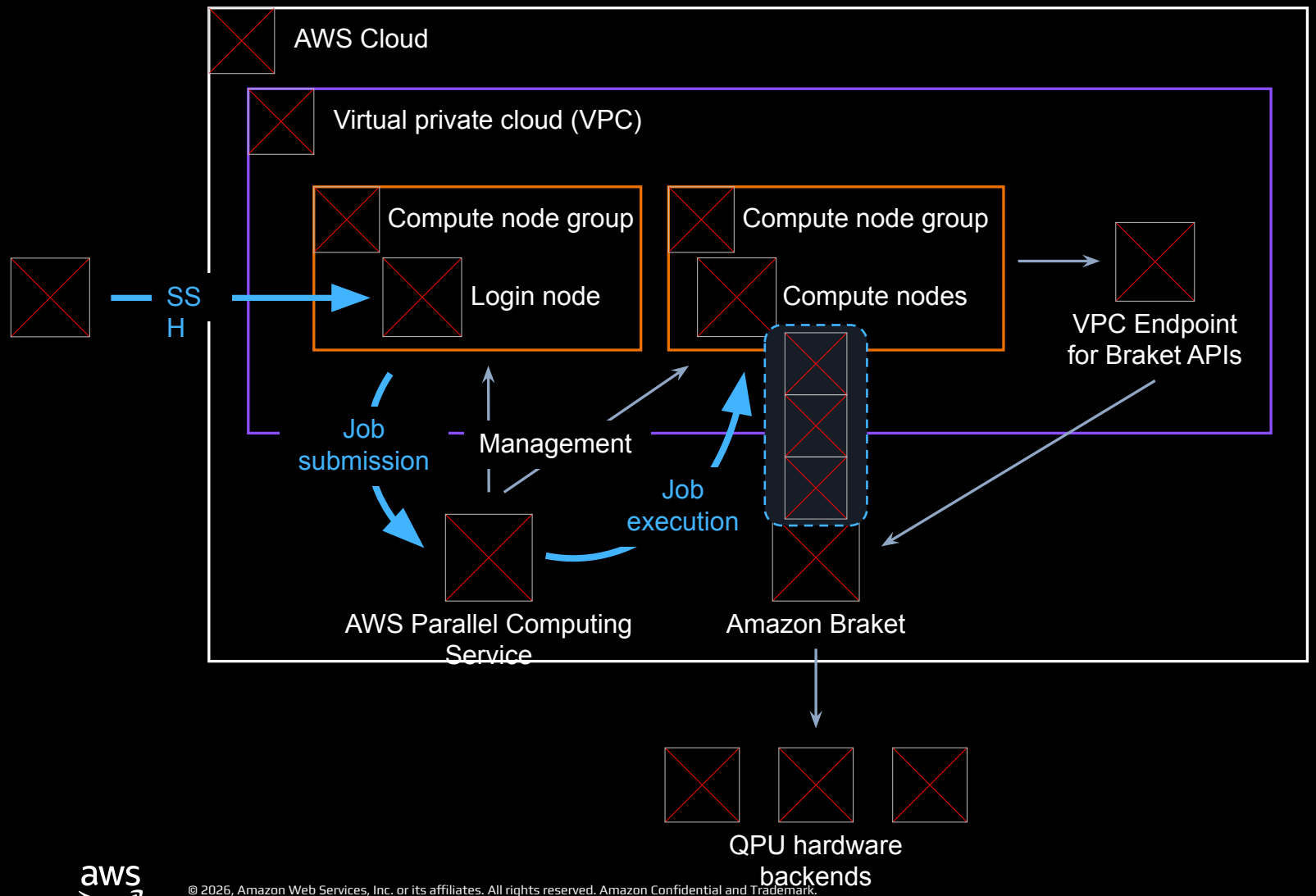
```
AWSTemplateFormatVersion: 2010-09-09
Resources:
  HybridWorkflowOrchestration:
    Type: AWS::StepFunctions::StateMachine
    Properties:
      RoleArn: !ImportValue BraketBatchWorkflowRoleArn
      Definition:
        QueryLanguage: JSONata
        StartAt: PassInputData
        States:
          BraketJob:
            Type: Task
            Resource:
arn:aws:states:::aws-sdk:braket:createJob.waitForTaskToken
            Arguments:
              AlgorithmSpecification:
                ScriptModeConfig:
                  EntryPoint: afqmc.collect_shadow:main
                  S3Uri: !Sub
s3://${DataBucket}/afqmc.tar.gz
                  CompressionType: GZIP
              DeviceConfig:
                Device: local:pennylane/lightning.qubit
              HyperParameters:
                TaskToken: "{% $states.context.Task.Token
%}"
```

- Describe workflow with code
- Deploy with an API call `CloudFormation:CreateStack`
- Run full simulation with an API call `StepFunctions:StartExecution`
- Download results from Amazon S3
- Develop, maintain, and share experiment including infrastructure setup and application code in one repository

Access to Braket from traditional HPC environments



Quantum-accelerated Slurm partitions in the cloud



Thoughts on resource scheduling

Option 1: Dedicated QPU access

- Device reservations give exclusive access to a QPU with exact start and end time
- Slurm resource reservations to reserve classical compute capacity in same timeframe

Option 2: On-demand QPU access

- Braket GetDevice API contains availability and queue depth information
- Spank plugin to configure job execution depending on QPU availability and utilization